



中国科学技术大学
University of Science and Technology of China

引 论

《编译原理和技术(H)》

张昱

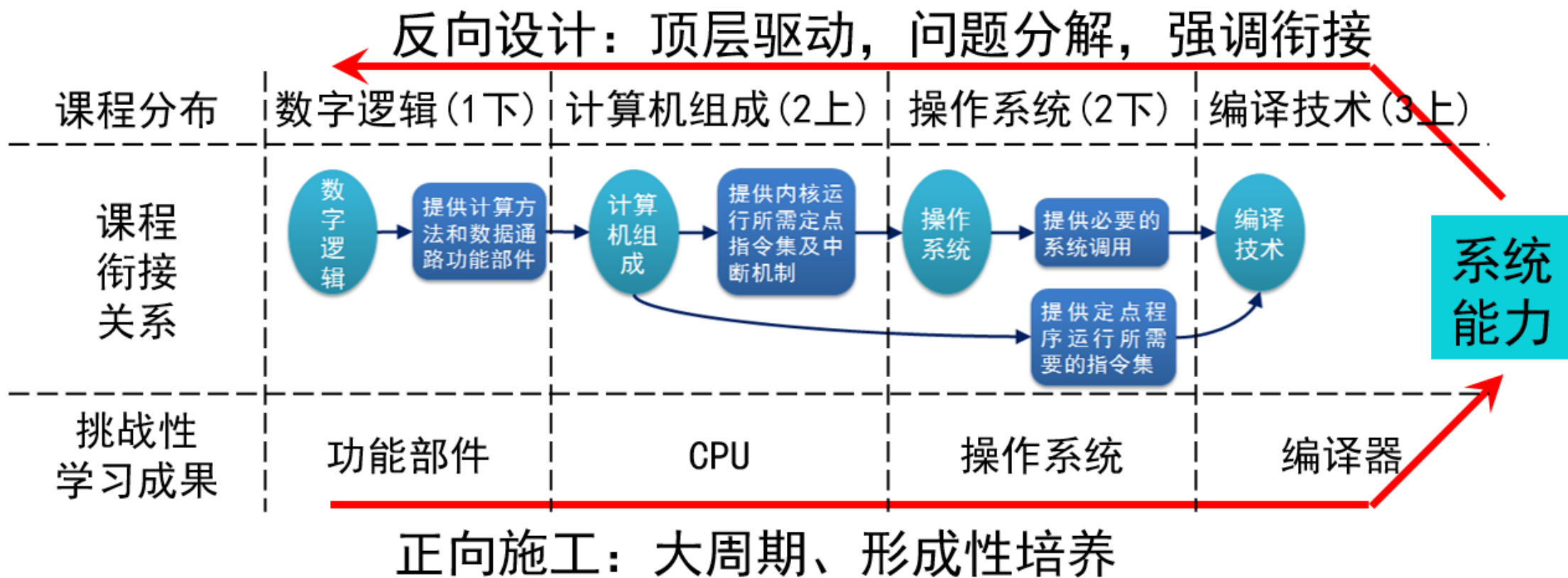
0551-63603804, yuzhang@ustc.edu.cn

中国科学技术大学
计算机科学与技术学院



编译课程：语言及实现、体系化实践

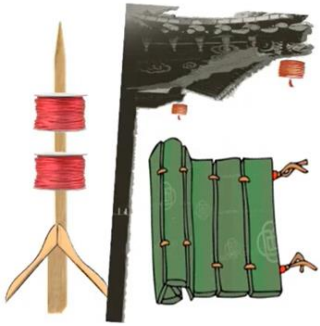
- 多门课程联动，整体设计，分布实施
 - 重视衔接：课程衔接有序，彼此有效支撑
 - 能力形成：实现系统能力的大周期形成性培养





编译：承古义而开新

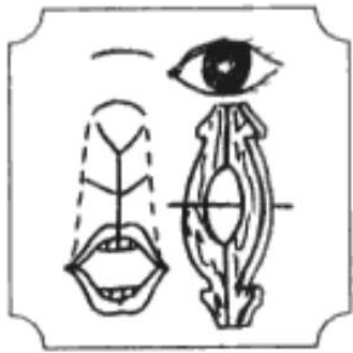
编



編

从糸，扁声；用丝线串联竹简成册

译



譯

从言，睪 (yì) 声；传译四夷之言者《说文》

将分散的元素按照一定的规则、顺序或结构组织起来，形成一个有内在逻辑的整体

理解和编排的质量
(结构合理、高效、节能)

将一种语言中的信息，用另一种语言等价地表达出来

保证从输入到输出的
语义正确性 (功能等价)

编译的本质： 在保持语义等价的前提下，对源语言程序进行结构分析与重组，并将其翻译为目标语言程序的过程。



ACM图灵奖

<https://amturing.acm.org/bysubject.cfm>

□ 编程语言、编译相关的获奖者是最多的 占约1/3

Analysis of Algorithms
Combinatorial Algorithms Compilers
Computer Architecture Computer Hardware
Data Structures Databases Education Error Correcting Codes Finite Automata Graphics
Interactive Computing Internet Communications List Processing Numerical Analysis
Numerical Methods Object Oriented Programming Operating Systems Personal Computing
Program Verification Programming
Programming Languages
Verification of Hardware and Software Models Computer Systems Machine Learning
Parallel Computation

Artificial Intelligence

Computational Complexity

Cryptography

Proof Construction Software
Theory Software Engineering





程序语言与编译系统发展的契机

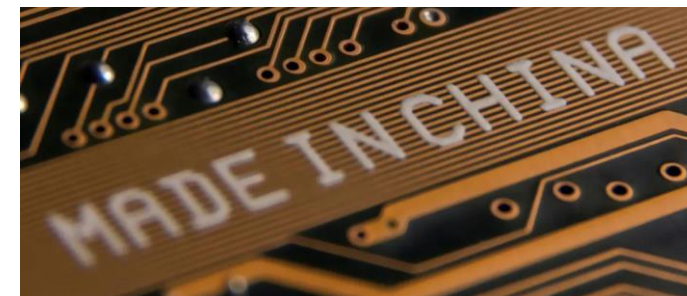
□ 人工智能的再次兴起，2021：人工智能的普及之年

- 人工智能加速芯片
- 人工智能算法开发

} 对程序语言与编译
提出更高要求

□ 国产芯片五年计划，2020年8月

- 到2025年将实现70%的芯片自给率 ??
- 2020年新增超过6万家芯片相关企业



➔ 面向应用/硬件的领域特定语言、软硬件协同的编译系统优化



程序语言与编译系统发展的契机

□ GPT、DeepSeek带来的影响

- 正成为软件开发、编译器优化和教育变革等的催化剂，但其广泛应用仍需解决可靠性、集成度和安全性问题。

Compilers and LLMs: Bidirectional Impact of Systems and Education

编译器和大型语言模型：系统与教育的双向影响

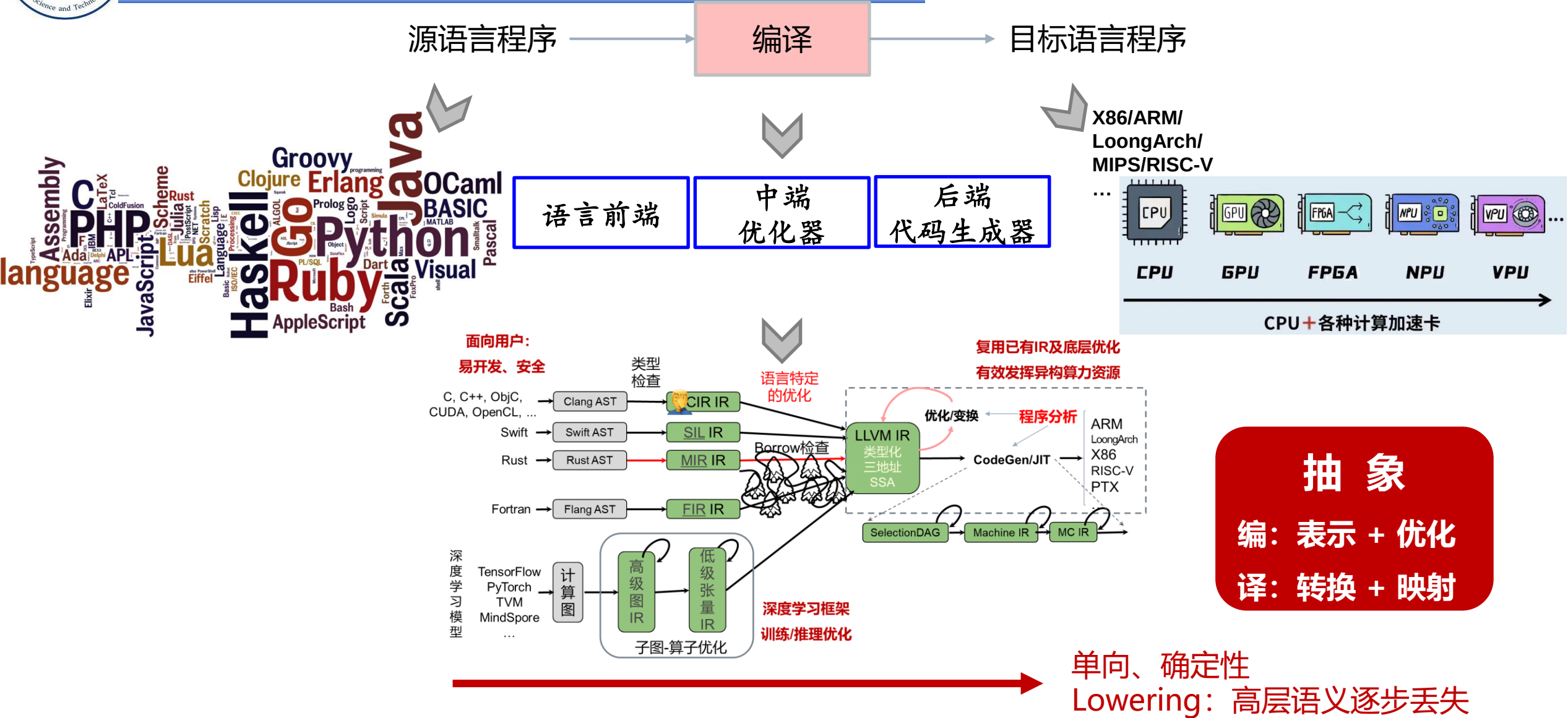
张昱 2025.7

摘要：大型语言模型（LLMs）正在深刻改变软件的开发与教学方式。与此同时，LLMs从根本上依赖于编译器的核心概念与技术基础。本文探讨了编译器系统与LLMs之间的双向影响——从LLMs如何重塑编译器的设计和使用，到编译器原理与技术如何为理解、构建和教学基于LLM的系统提供关键支撑。我们进一步审视其对软件工程教育的启示，并就如何将LLMs融入未来编译器课程提出初步构想。通过桥接传统编译器基础与新兴AI范式，我们主张重新确立编译器教育在培养下一代智能系统开发者中的核心地位。

张昱 等. [Compilers and LLMs: Bidirectional Impact of Systems and Education](#). 第21届中欧软件工程教育国际会议, 中国杭州, Sep.20-21, 2025.



经典编译的演进



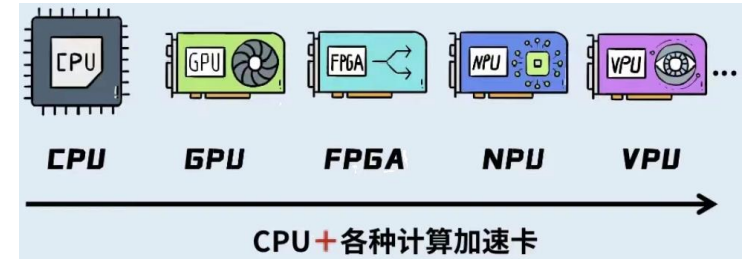
LLM时代的编译



源语言程序

编译
+ 自动调优

目标语言程序



编码

生成式AI
+
Harness

生成器

检查器 (Agentic)

知识+学习+推理+预测+对齐

抽象

编：表示 + 优化 + 智能降阶

译：转换+映射+语义保留生成

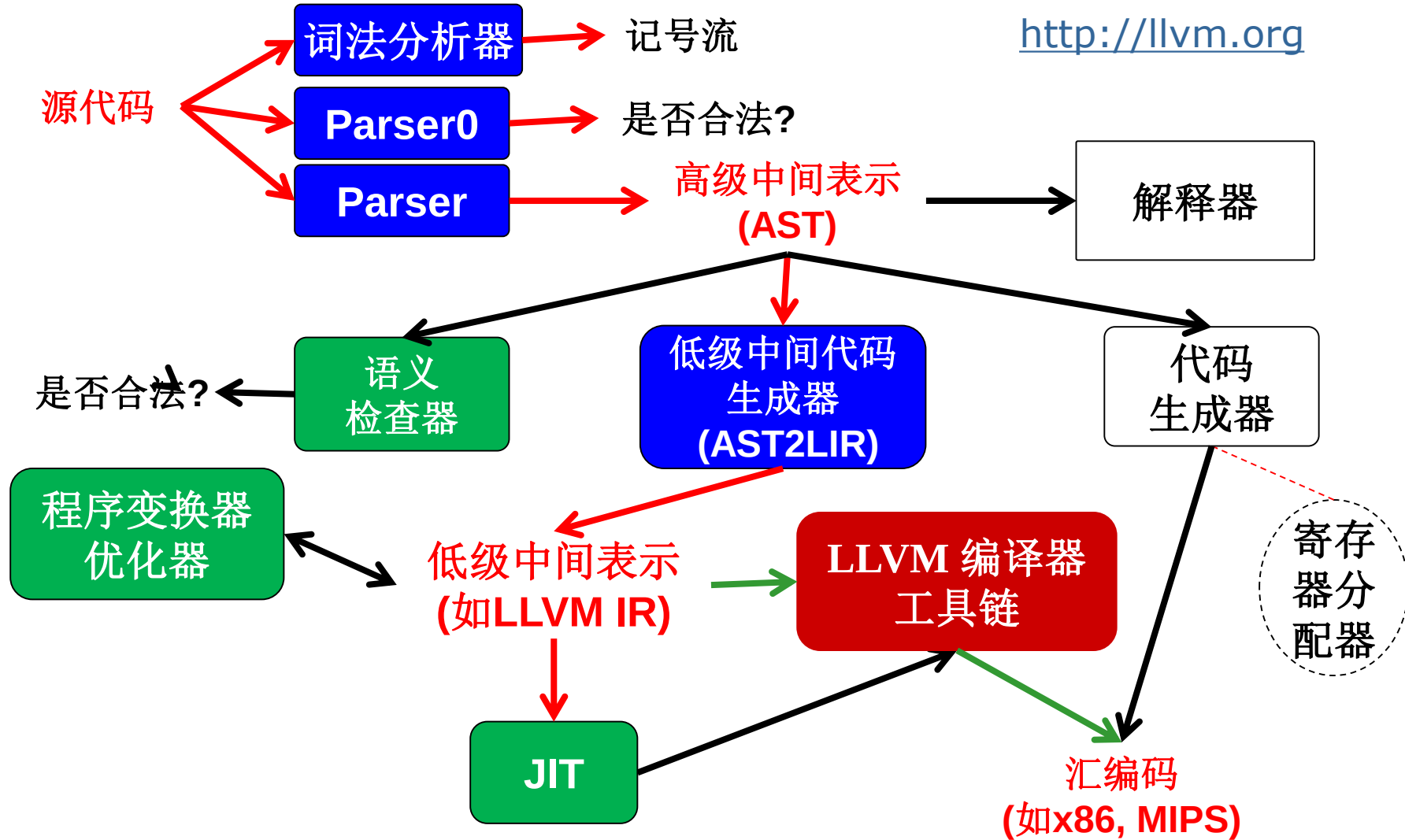
双向、可逆（可推测高层语义）、概率性生成



毕昇杯编译系统设计赛(2020-2026)

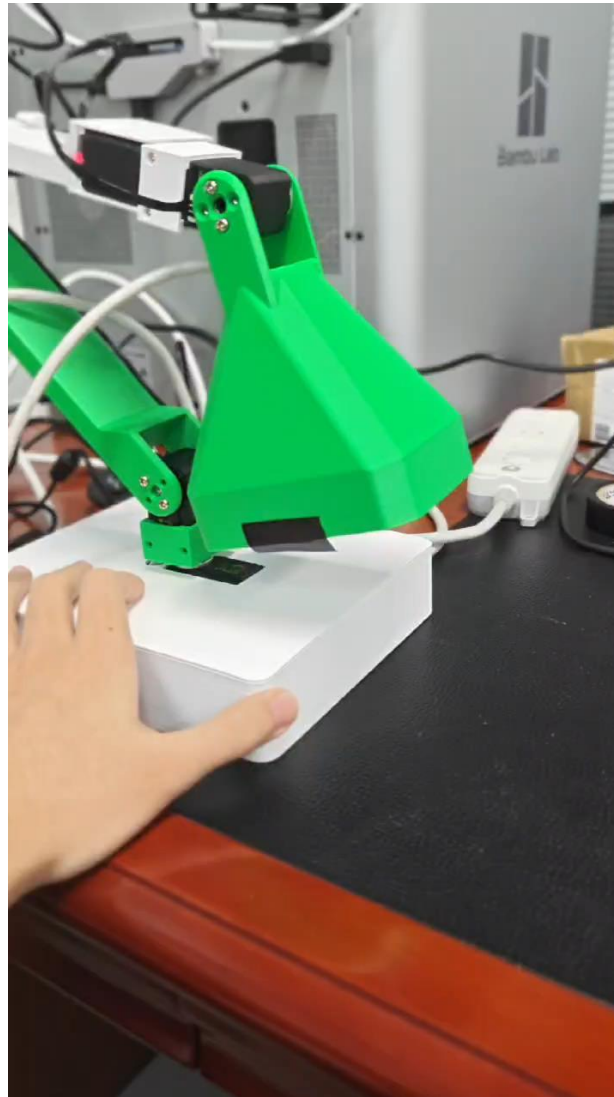
- 累计959支参赛队报名参赛，累计参赛学生约3200人
- 累计覆盖30余个省（市/自治区）的百余所高校（尚未覆盖到西藏、宁夏、台湾、澳门）
- 吸引来自英国、加拿大、巴西、俄罗斯等地的9所海外高校参与

2020	2021	2022	2023	2024	2025	2026
报名队伍 72	报名队伍 95	报名队伍 151	报名队伍 160	报名队伍 184	报名队伍 185	报名队伍 297
取得有效成绩 31	取得有效成绩 33	取得有效成绩 53	取得有效成绩 69	取得有效成绩 75	取得有效成绩 89	取得有效成绩 216
	进入决赛队伍 25+4 外卡	进入决赛队伍 30+12 外卡	进入决赛队伍 54+12 外卡	进入决赛队伍 49+17 外卡	进入决赛队伍 60+11 外卡	进入决赛队伍 110+23 外卡
决赛高校数量 15	决赛高校数量 18	决赛高校数量 21	决赛高校数量 23	决赛高校数量 28	决赛高校数量 33	决赛高校数量 40





智能灯：理想与现实



<https://www.l Lamp.com/>

<https://github.com/humancomputerlab/LeLamp>

张昱：《编译原理和技术(H)》引论



主要内容

1

编程语言及设计

2

编译器的阶段

3

编译器的作用及形式

4

编译技术的应用与挑战



主要内容

1

编程语言及设计

2

编译器的阶段

3

编译器的作用及形式

4

编译技术的应用与挑战



□ 什么是编程语言

- **A programming language** is a notation for describing computations to people and to machines.

□ 每种编程语言有自己的计算模型

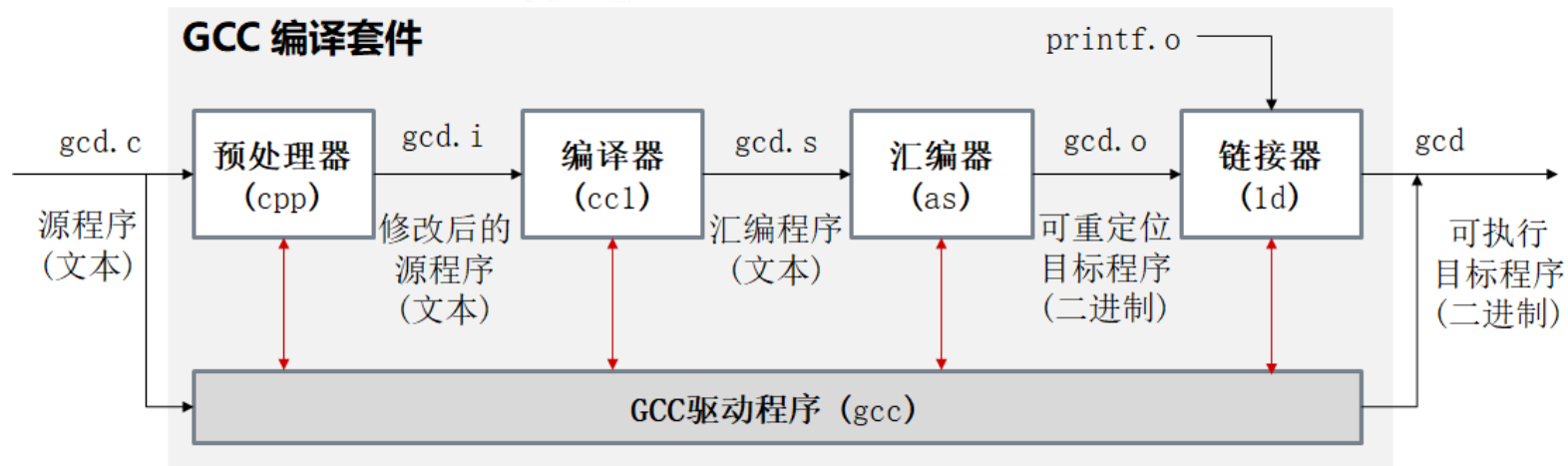
- 过程型(Procedural): **C, C++, C#, Java, Go**
- 声明型(Declarative): **SQL, ...**
- 逻辑型(Logic): **Prolog, ...**
- 函数式(Functional): **Lisp/Scheme, Haskell, ML, OCaml...**
- 脚本型(Scripting): **AWK, Perl, Python, PHP, Ruby, ...**



求最大公约数 gcd

```
int gcd(int a, int b) {  
    while (a != b) {  
        if (a > b) a = a - b;  
        else b = b - a;  
    }  
    return a;  
}
```

// C



```
let rec gcd a b =  
    if a = b then a  
    else if a > b then gcd b (a - b)  
    else gcd a (b - a)
```

(* OCaml *)

```
gcd(A,B,G) :- A = B, G = A.  
gcd(A,B,G) :- A > B, C is A-B, gcd(C,B,G).  
gcd(A,B,G) :- B > A, C is B-A, gcd(C,A,G).
```

% Prolog



□ <https://godbolt.org/>

The screenshot displays the Godbolt online compiler interface with four main panels:

- A. 源代码 (Source Code):** Shows Python source code for a GCD function and its usage. A red arrow points to the language dropdown menu, labeled "选择编程语言" (Select programming language).
- B. 编译: 汇编码 (Compile: Assembly):** Shows the compiled assembly code for Python 3.12. A red arrow points to the compiler dropdown menu, labeled "选择编译器" (Select compiler).
- C. 运行输出 (Run Output):** Shows the execution output of the program, displaying "gcd(21,36)=3".
- D. LLVM IR输出 (LLVM IR Output):** Shows the LLVM IR output, which is currently empty.

□ <https://onecompiler.com/>

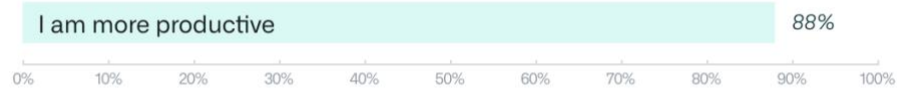


编程语言发展：2022-AI辅助编码

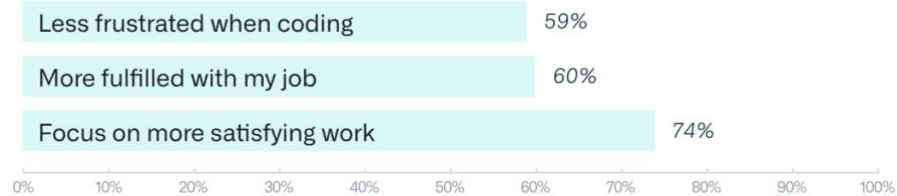
□ Research: quantifying GitHub Copilot's impact on developer productivity and happiness

When using GitHub Copilot...

Perceived Productivity



Satisfaction and Well-being*



Efficiency and Flow*



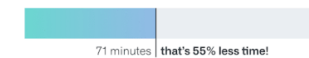
We recruited
95
developers, and split them randomly into two groups.

We gave them the task of writing a web server in JavaScript

45 Used
GitHub Copilot

78%
finished

1 hour, 11 minutes
average to complete the task



50 Did not use
GitHub Copilot

70%
finished

2 hours, 41 minutes
average to complete the task



Results are statistically significant ($P=0.017$) and the 95% confidence interval is [21%, 89%]

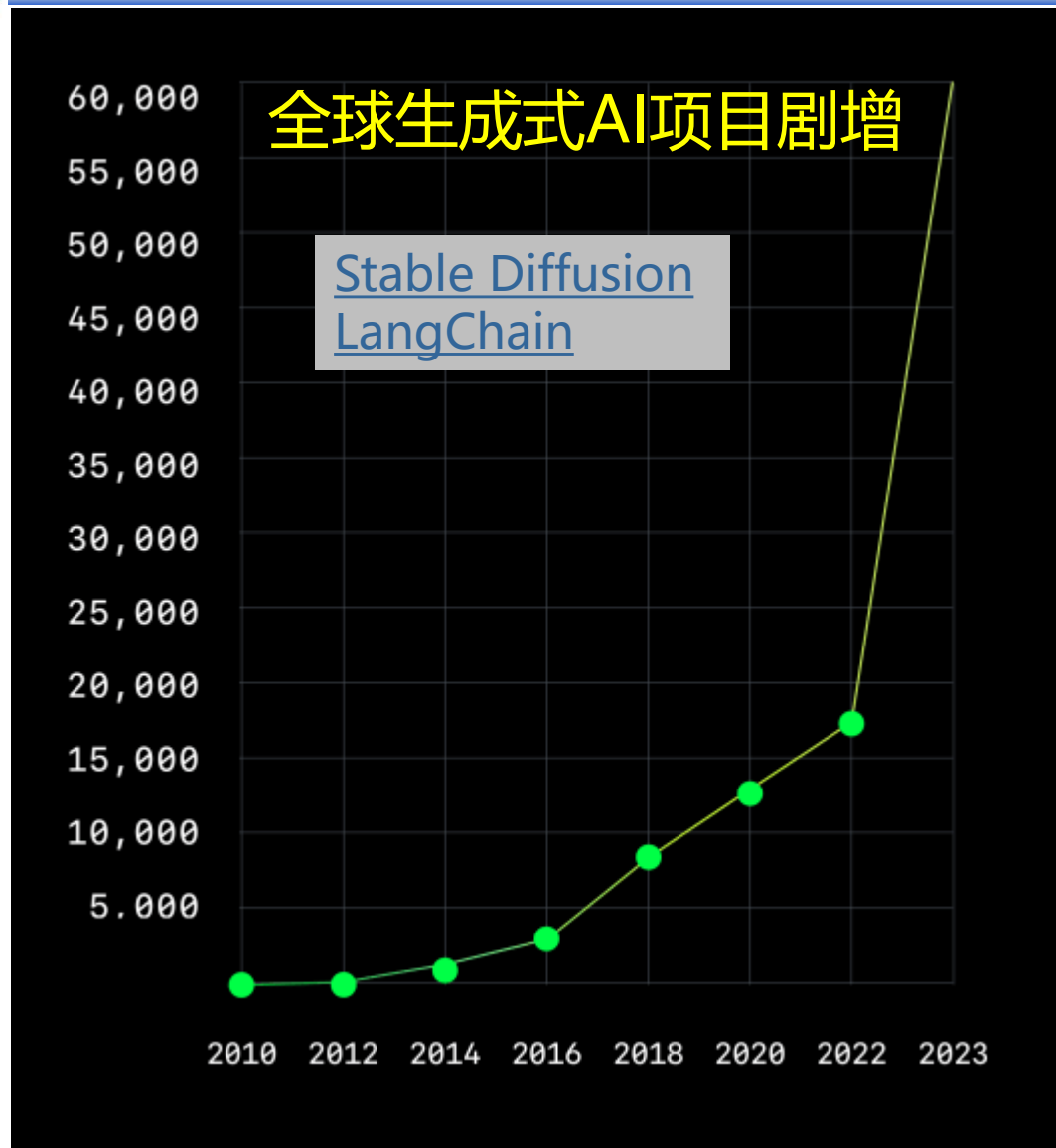
an evaluation with 24 students,

Google's internal assessment of
ML-enhanced code completion

[ACMQueue202109] The SPACE of Developer Productivity



编程语言发展2023-生成式AI、类型安全↑



□ 类型安全

■ TypeScript (2012~) 渐进类型

首次超越Java

第三受欢迎的语言

其用户群增长了 37%

集语言、类型检查器、编译器和语言服务于一体

■ Rust (2009~) 所有权

[用于系统编程及纳入Linux内核的评论](#)

年使用增长率为 40%

被 2023 年 Stack Overflow 开发者调查
评为最受推崇的语言

程序语言设计的计算思维

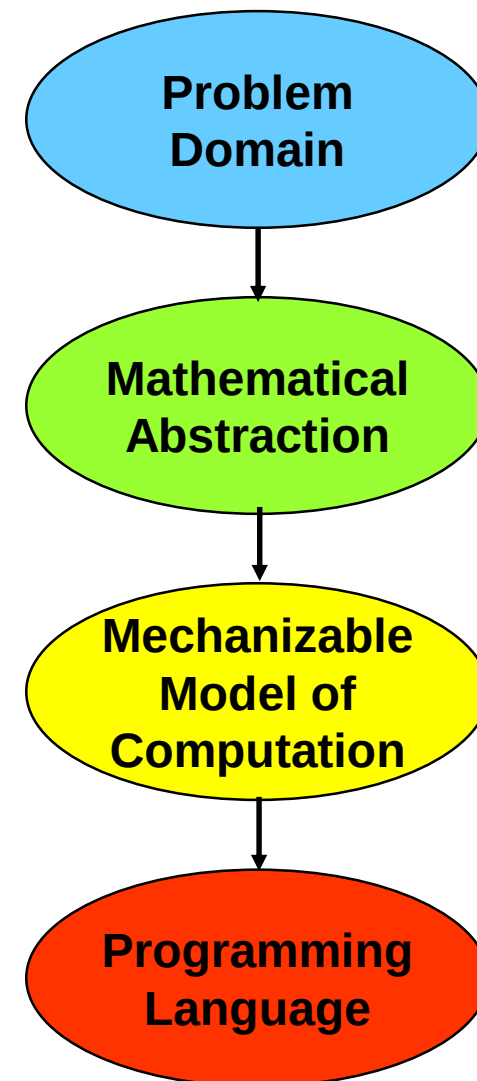
□ 计算思维 (Computational Thinking)



Jeannette M. Wing
Computational Thinking
CACM, vol. 49, no. 3, pp. 33-35, 2006

Computational thinking is a **fundamental skill for everyone**, not just for computer scientists. To reading, writing, and arithmetic, **we should add computational thinking to every child's analytical ability**. Just as the printing press facilitated the spread of the three Rs, what is appropriately incestuous about this vision is that computing and computers facilitate the spread of computational thinking.

□ 语言设计中的计算思维





主要内容

1

编程语言及设计

2

编译器的阶段

3

编译器的作用和形式

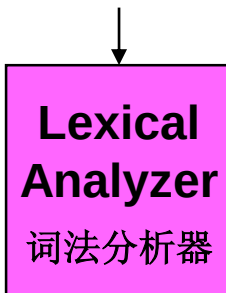
4

编译技术的应用与挑战



编译器的阶段

source
program



Token
Stream
记号流

□ 词法分析：将程序字符流分解为记号
(Token) 序列

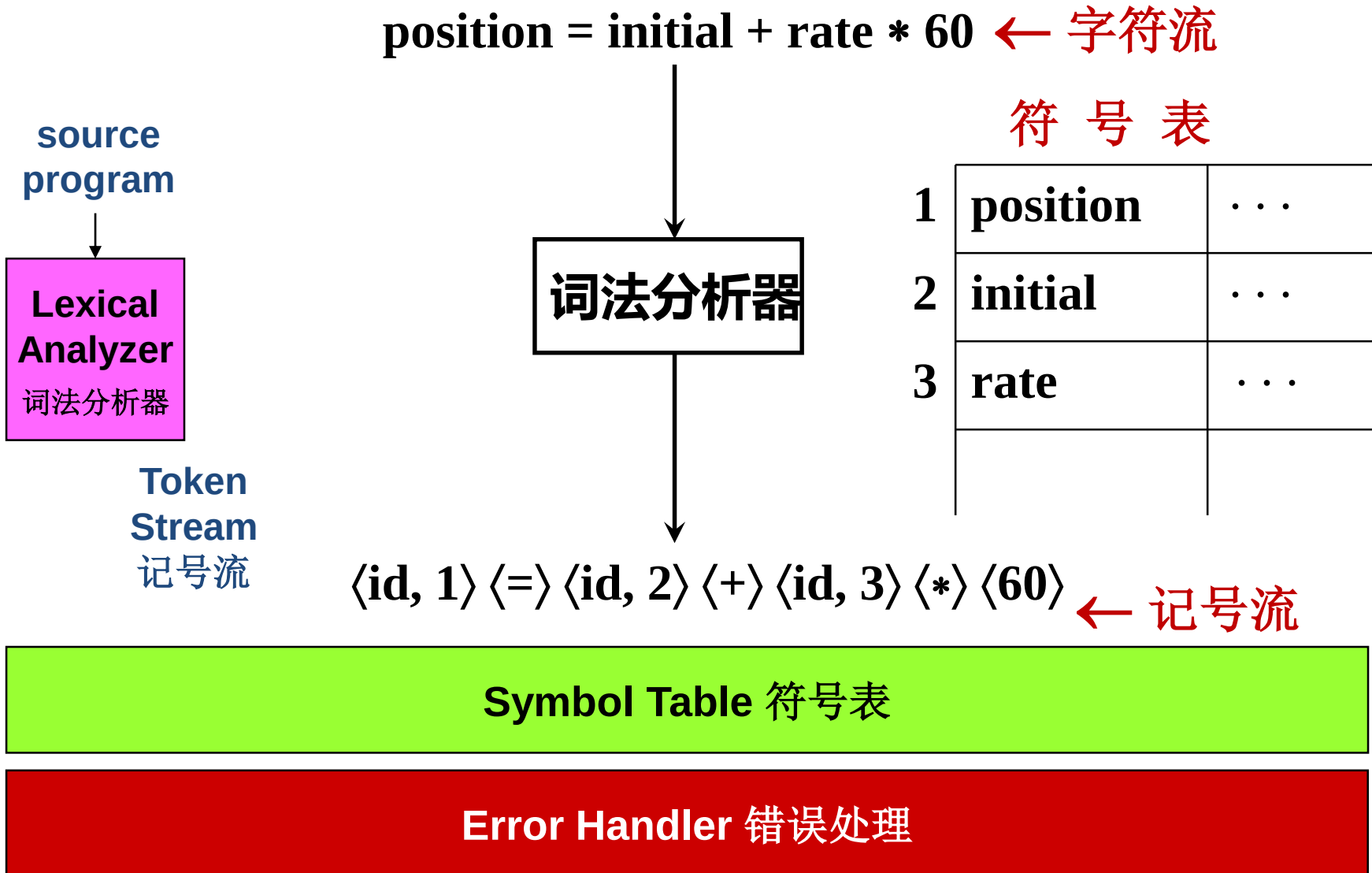
◆ 形式：<token_name, attribute_value>

Symbol Table 符号表

Error Handler 错误处理

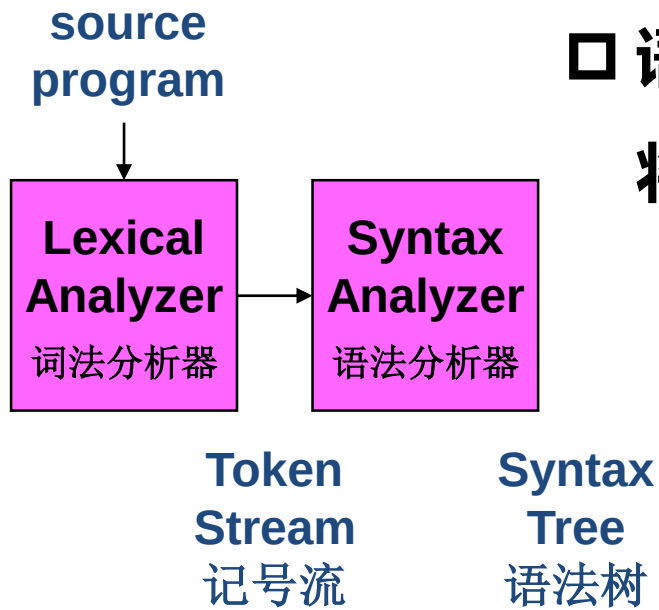


编译器的阶段





编译器的阶段



□ 语法分析：也称解析(Parsing)

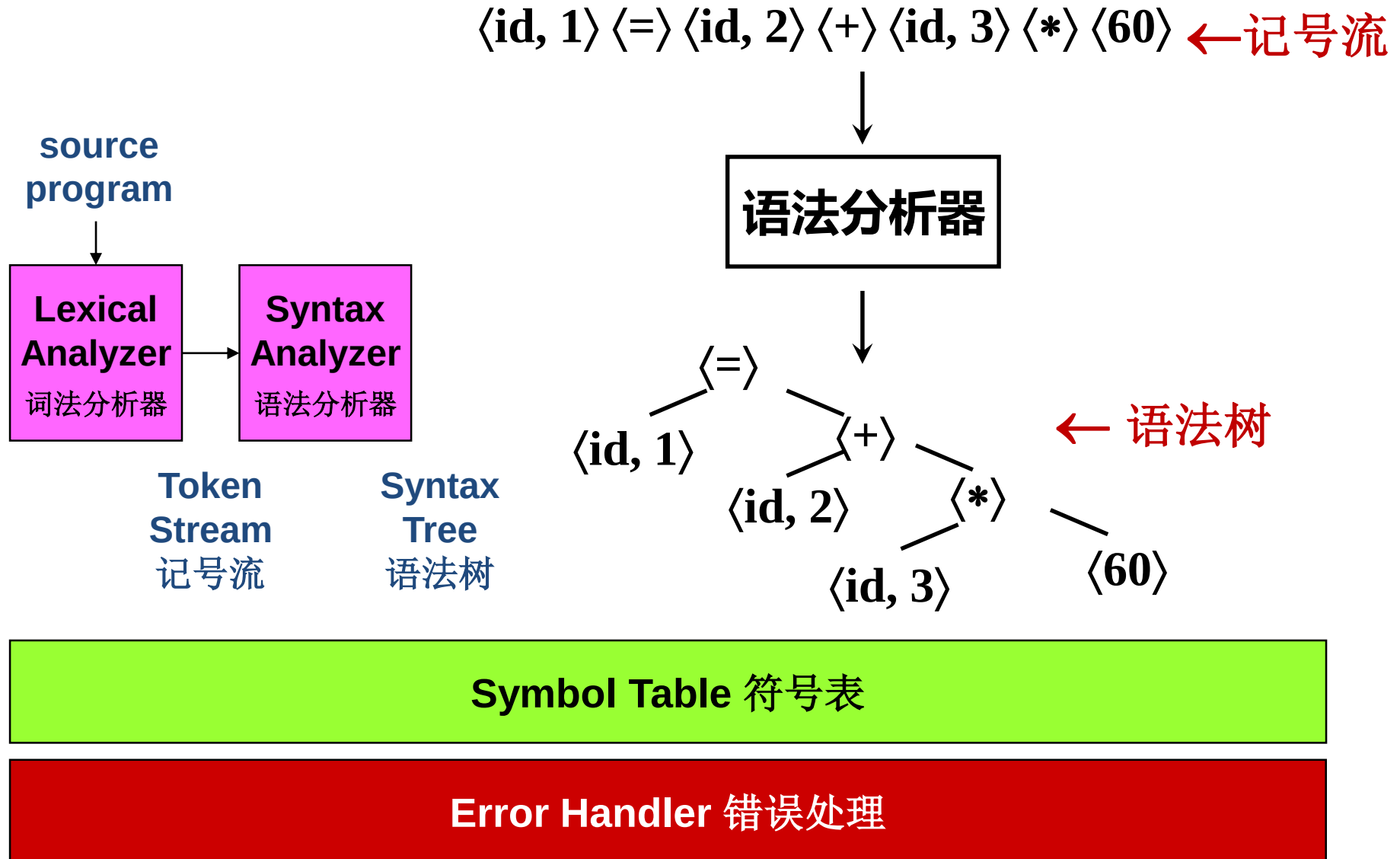
将记号序列解析为语法结构

Symbol Table 符号表

Error Handler 错误处理

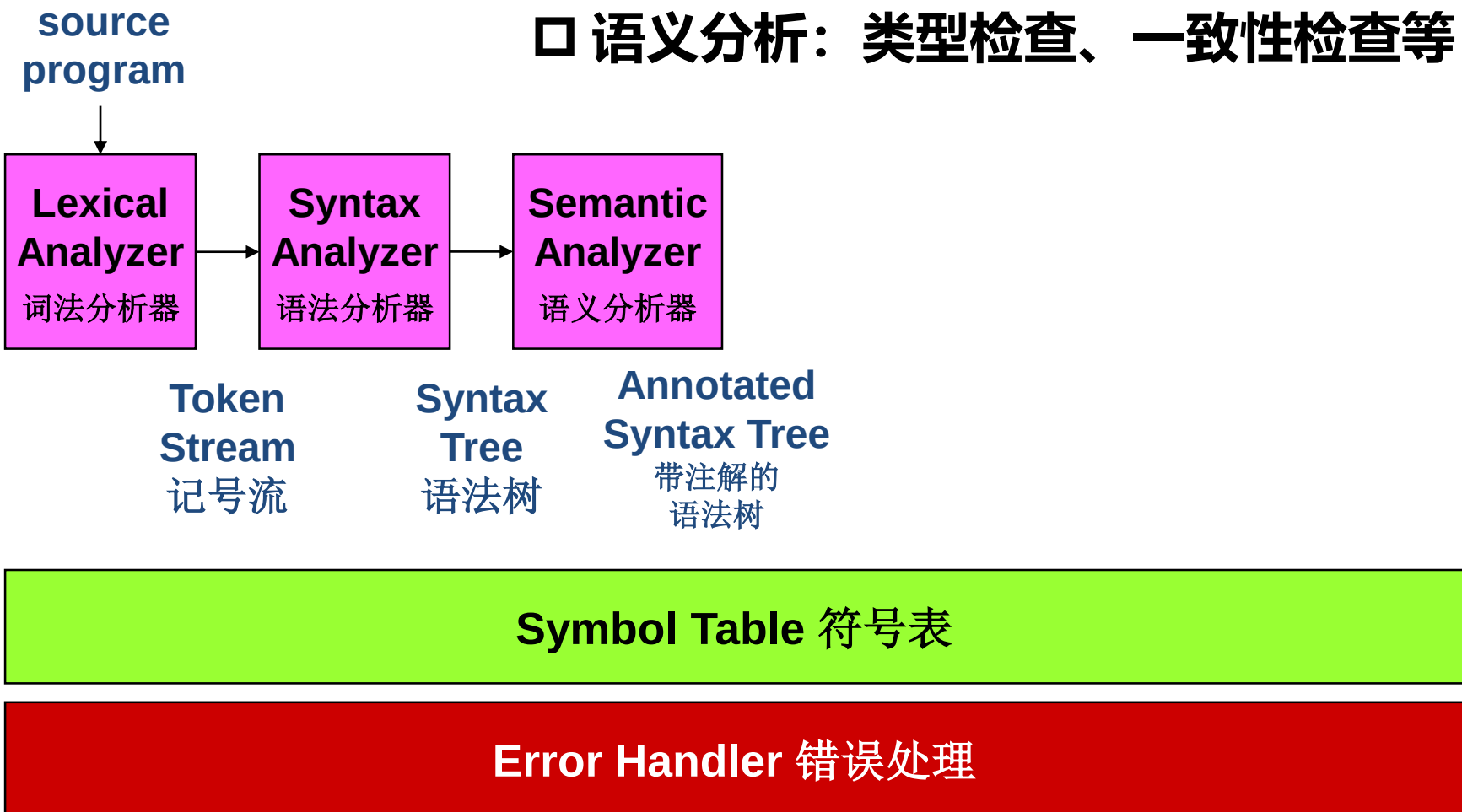


编译器的阶段

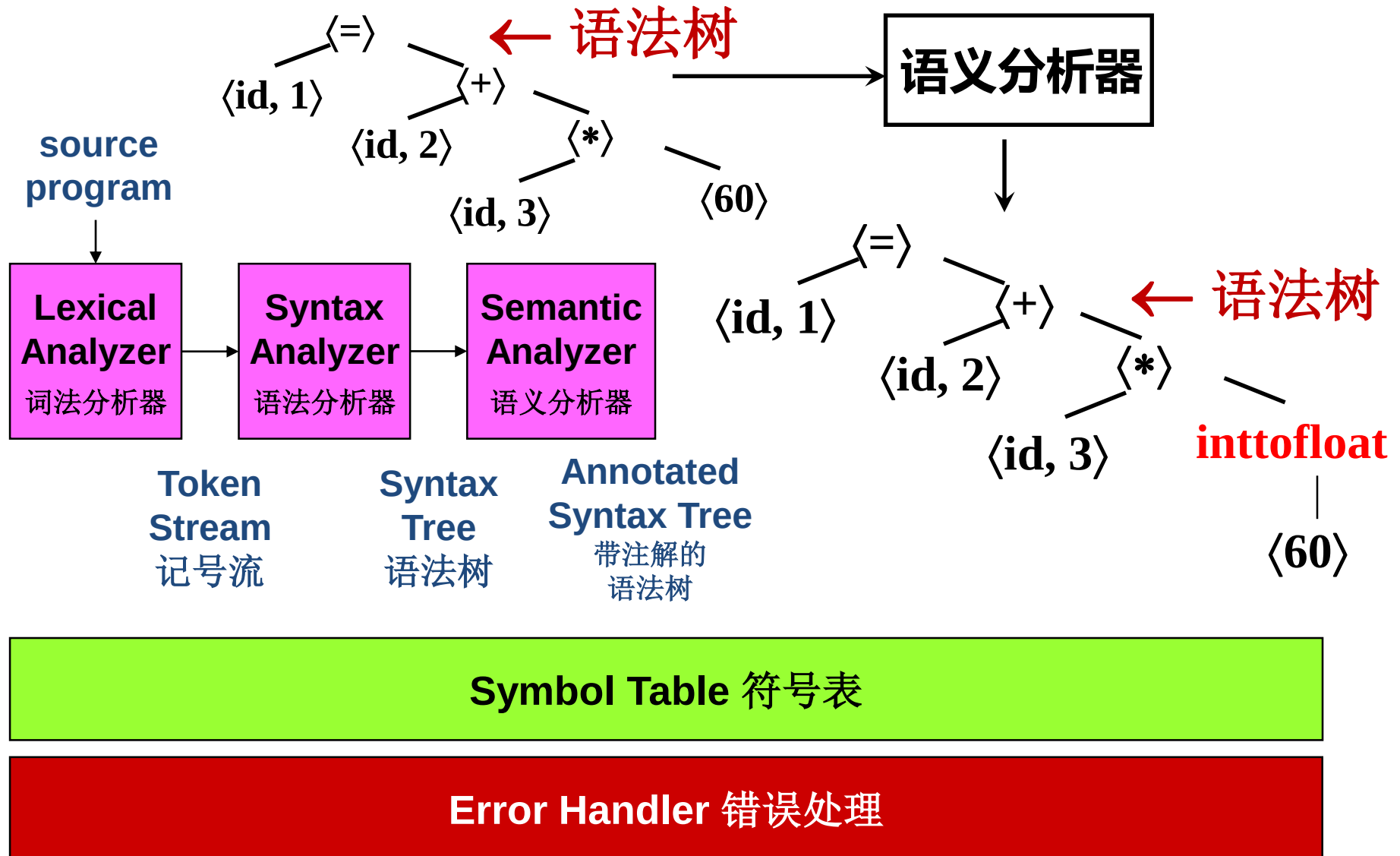




编译器的阶段



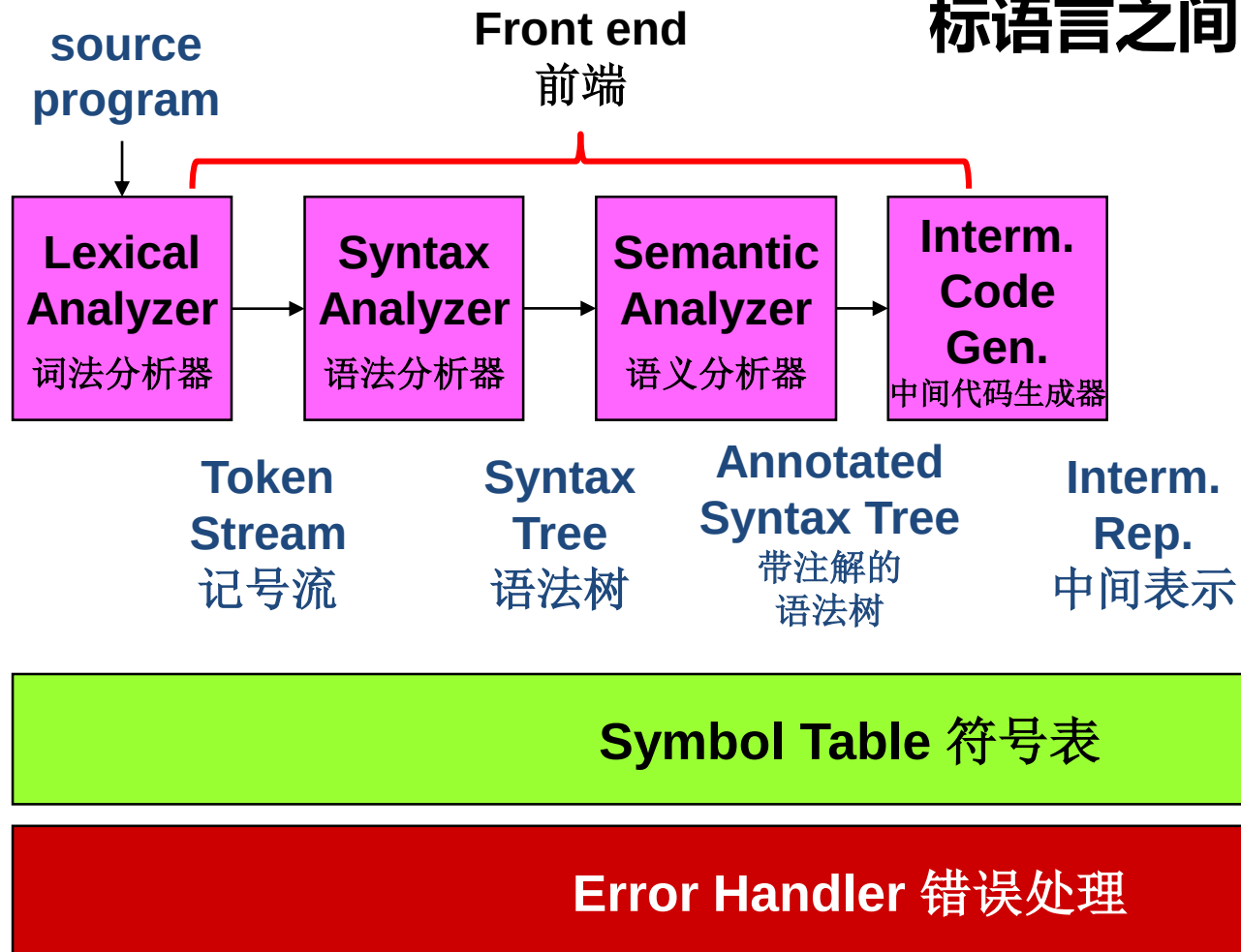
编译器的阶段





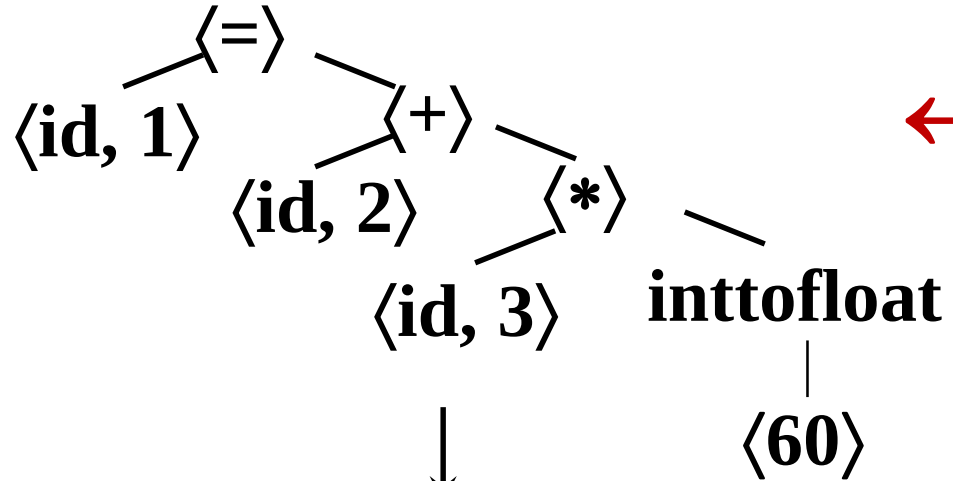
编译器的阶段

□ 中间代码生成：源语言与目标语言之间的桥梁





编译器的阶段



← 语法树

符号表

1	position	...
2	initial	...
3	rate	...

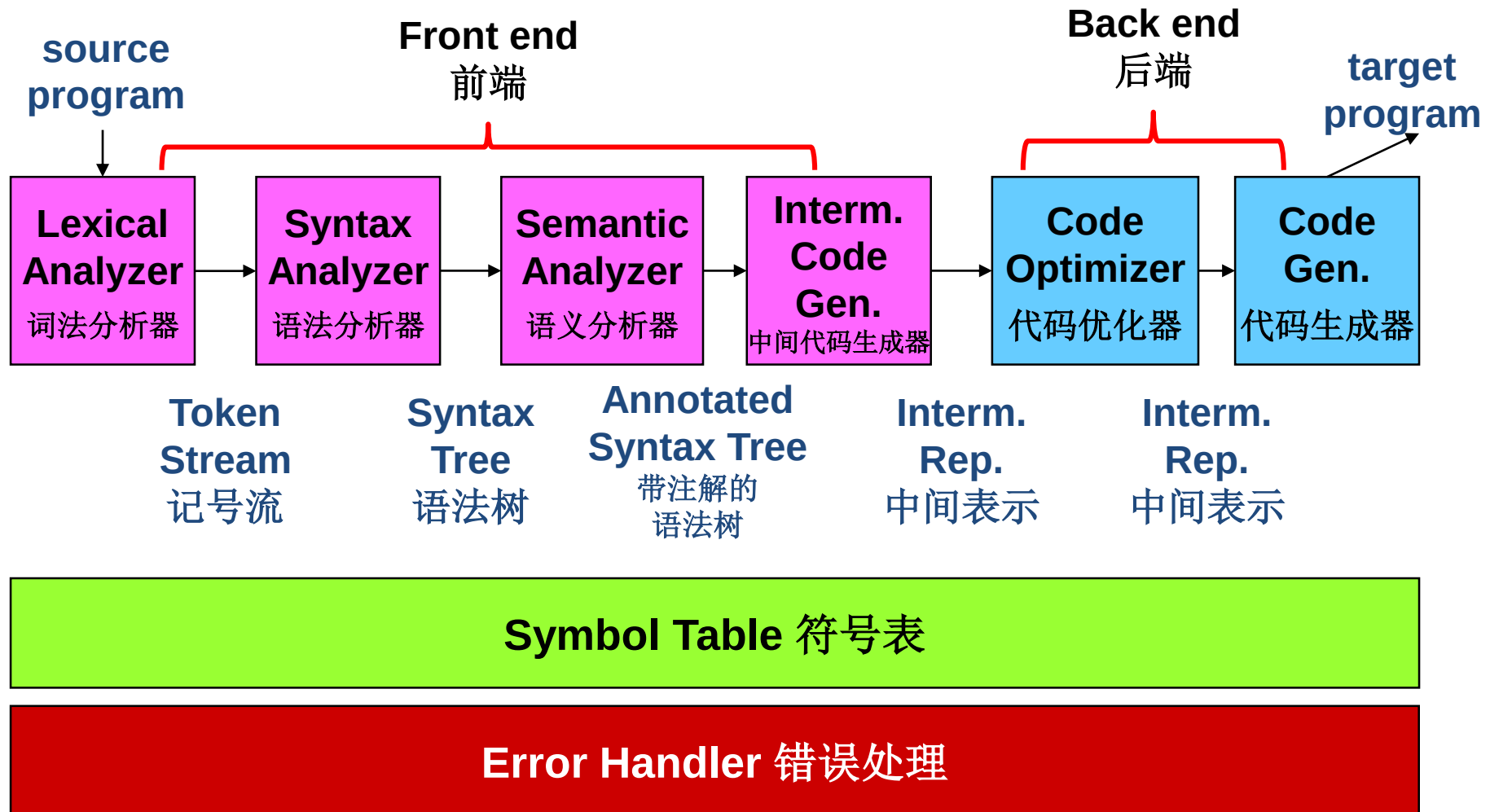
中间代码生成器

t1 = inttofloat(60)
t2 = id3 * t1
t3 = id2 + t2
id1 = t3

← 三地址中间代码



编译器的阶段





代码优化

- 机器无关的优化、机器相关的优化
- 降低执行时间，减少能耗、资源消耗等

t1 = inttofloat(60) ← 三地址中间代码

t2 = id3 * t1

t3 = id2 + t2

id1 = t3



代码优化器



t1 = id3 * 60.0

id1 = id2 + t1

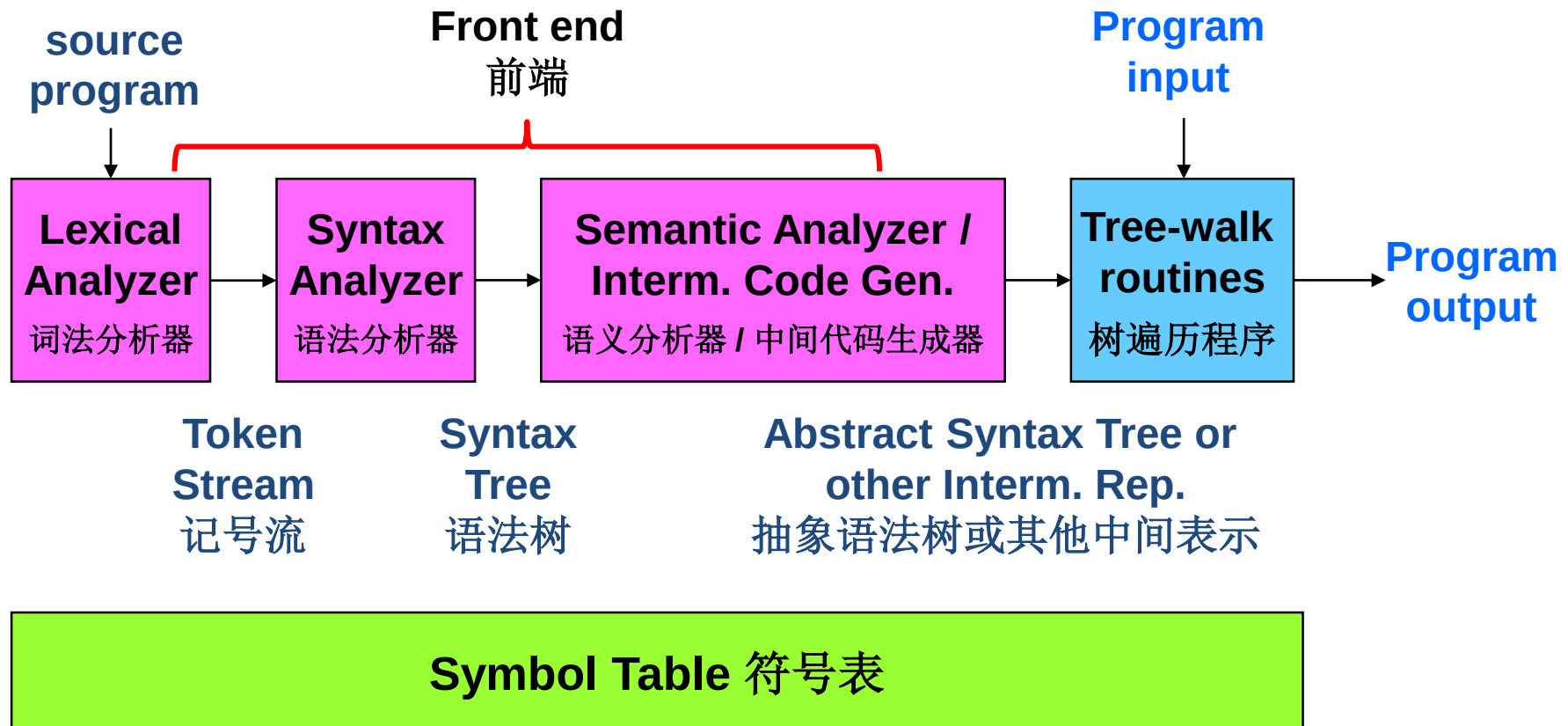
← 三地址中间代码

符号表

1	position	...
2	initial	...
3	rate	...



编译器的阶段





主要内容

1

编程语言及设计

2

编译器的阶段

3

编译器的作用和形式

4

编译技术的应用与挑战



编译器的作用

□ 编程语言

```
#include <stdio.h>
int main()
{
    printf("hello, world!\n");
}
```

□ 目标机器语言

```
/* helloworld.c */
```

□ 编译器（编译系统）

- 主流的开源编译基础设施：[GCC](#)、[Clang/LLVM](#)

```
[root@host ~]# gcc helloworld.c -o helloworld
```

```
[root@host ~]# ./helloworld
```

```
hello, world!
```

注意：gcc是驱动程序
(根据命令行参数调用相应的处理程序)



编译器的作用

□ 翻译

- 支持高层的编程抽象
- 支持底层的硬件体系结构

□ 优化

- 更快的执行速度
- 更少的空间

□ 分析

- 程序理解
- Safety: 自身的稳定状态, 功能正确
- Security: 免受外部伤害



举例：性能与安全

```
for (i=0; i<n; i++) a[i] = 1;
```

```
pend = a+n;  
for (p=a; p<pend; p++) *p = 1;
```

哪个更快, Why?

```
foo (char * s)  
{  
    char buf[32];  
    strcpy (buf, s);  
}
```

调用foo()会如何?



举例：性能与安全

```
for (i=0; i<n; i++) a[i] = 1;
```



```
pend = a+n;  
for (p=a; p<pend; p++) *p = 1;
```

哪个更快, Why?

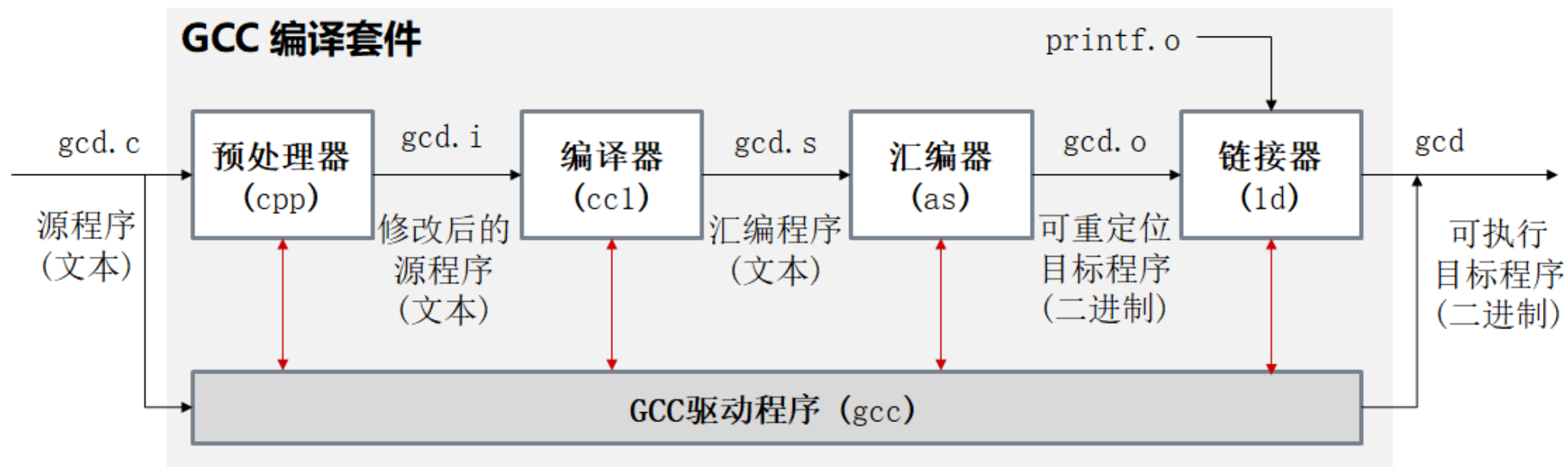
```
foo (char * s)  
{  
    char buf[32];  
    strcpy (buf, s);  
}
```

调用foo()会如何?

若s指向的串的长度超出31, 则复制时会超出buf数组的有效区域



□ GCC（GNU编译套件）：1987~



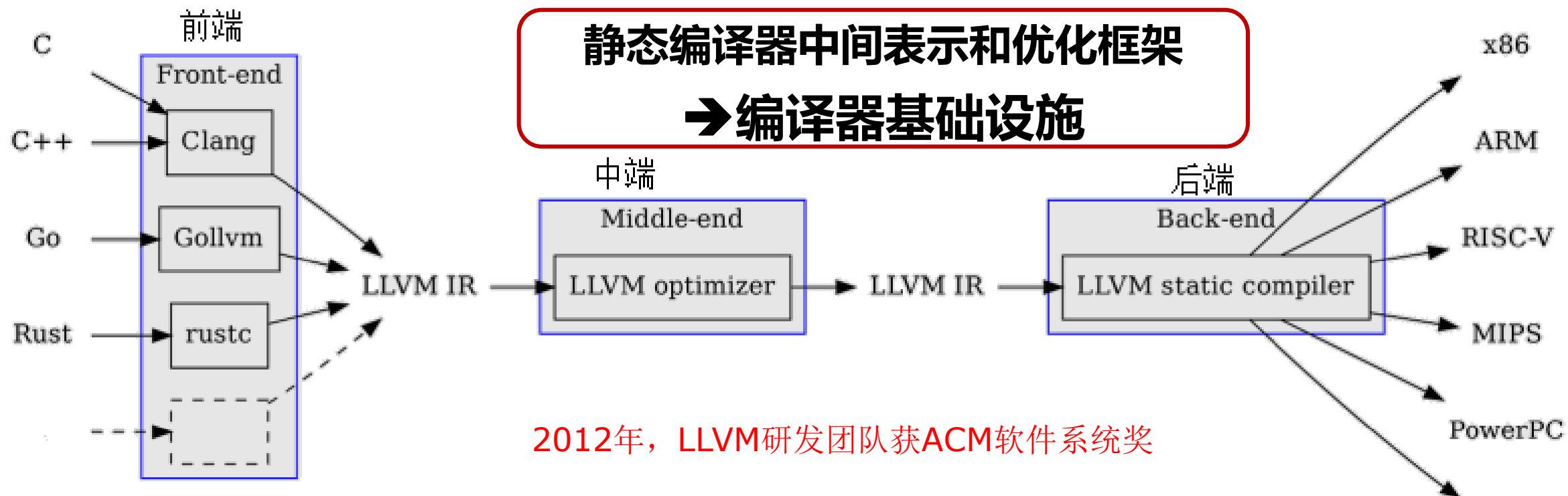
■ 支持多种编程语言、多种硬件架构 2014 年 GCC 获 ACMSIGPLAN 编程语言软件奖

■ 遵循 GNU General Public License (GPL) 许可

- 一些组件可能使用 GPL v2：强调软件自由和源代码的可用性
- 大多遵循 GPL v3：进一步增加对专利侵权的保护，防止 Tivoization

Tivoization: 诸如制造商提供了源代码但却不允许自由修改这个软件等硬件限制

□ **Low-level Virtual Machine:** Chris Lattner于2000年在UIUC创建

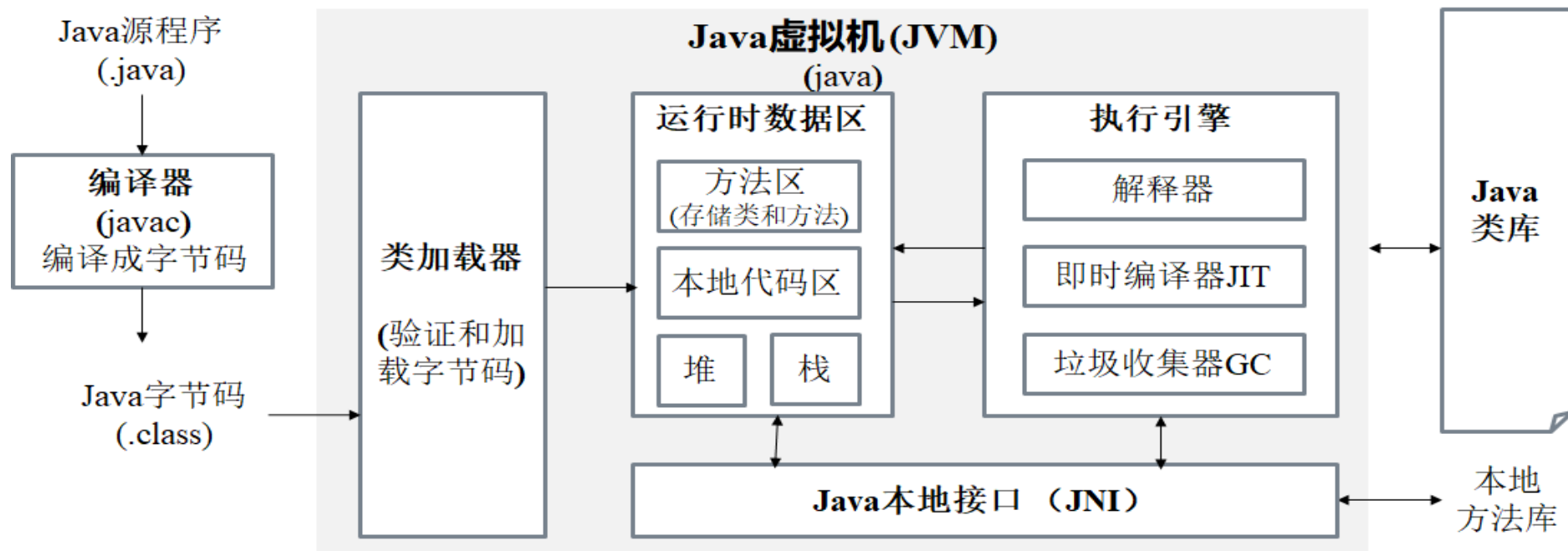


- **模块化、高级优化、广泛的应用领域**（研发编译器、语言、HPC和嵌入式等）
- **遵循Apache License 2.0 许可：** 自由使用、修改和分发软件，无需支付费用。
必须保留原始许可声明、版权声明和免责声明



Java编译运行时系统：Java虚拟机

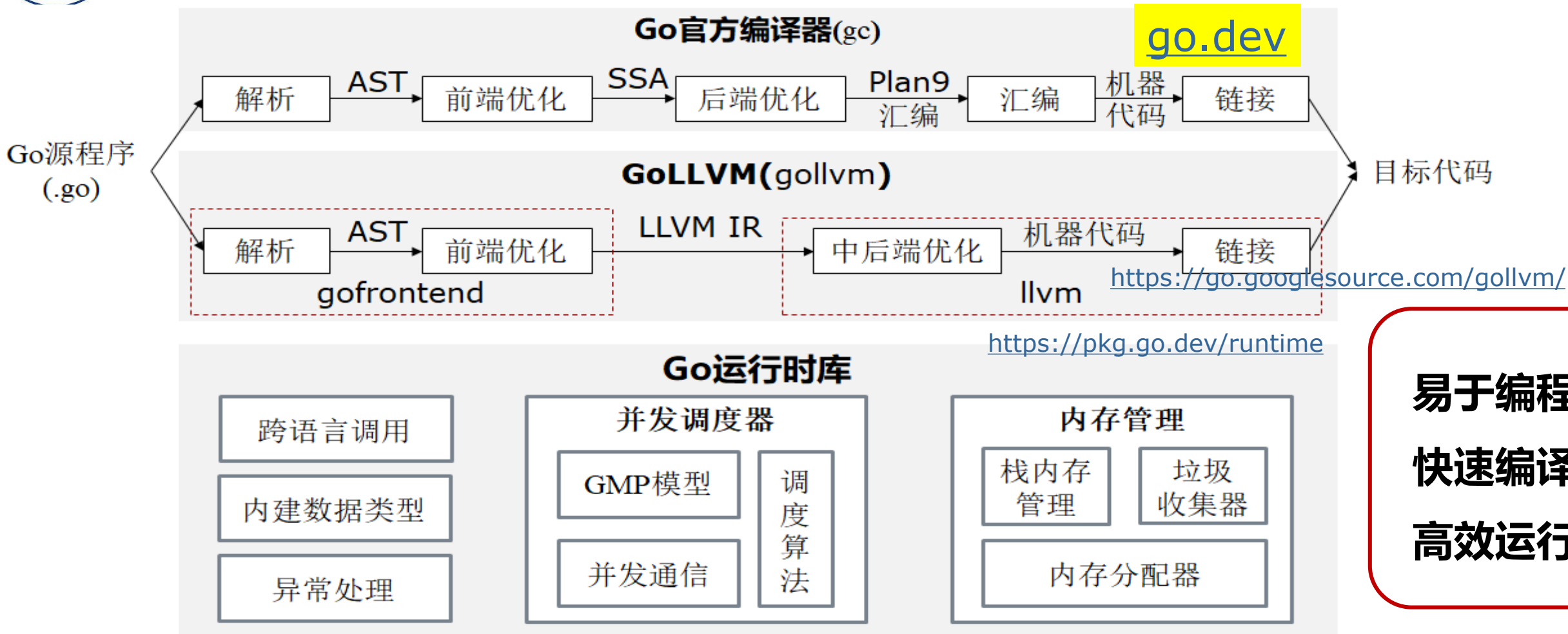
□ Java语言：1995~



■ 平台无关的字节码，即时编译JIT、垃圾收集GC

■ 不同的JVM遵循的许可不同

□ OpenJDK遵循 GPL v2 许可，并附加 Classpath Exception，允许将 GPL 代码与非 GPL 代码链接，减少对其他开源许可的限制。



- **遵循Go语言许可(基于BSD许可扩展)**：允许自由使用、修改和分发Go的源代码，适用于商业和非商业项目



国内编译技术生态

- 主要基于GCC和LLVM扩展：龙芯、申威、华为等
- 以华为编译技术发展为例

编译器团队成立

- 第一批海内外研究人员加入

无线DSP编译器业界领先

- DSP编译器性能超越标杆XCC，基站竞争力超越E
- CPU/DSP编译器规模商用，在网规模超xxw

ARM64编译器竞争力领先

- 自研ARM C/C++编译器实现ARM64 领域竞争力领先，并在华为公司多个场景商用；
- DSP编译器支撑基站、控制器等海量发货xx万套，服务全球xx亿用户

计算/AI等新领域竞争力突破

- 发布**毕昇编译器**，实现鲲鹏原生性能提升和多样算力融合优化
- 发布二进制翻译工具ExaGear，实现x86到ARM生态平滑迁移
- 昇腾编译器完整支持ISA6.3与V100全系列芯片，助力Atlas集群挺进秒级训练



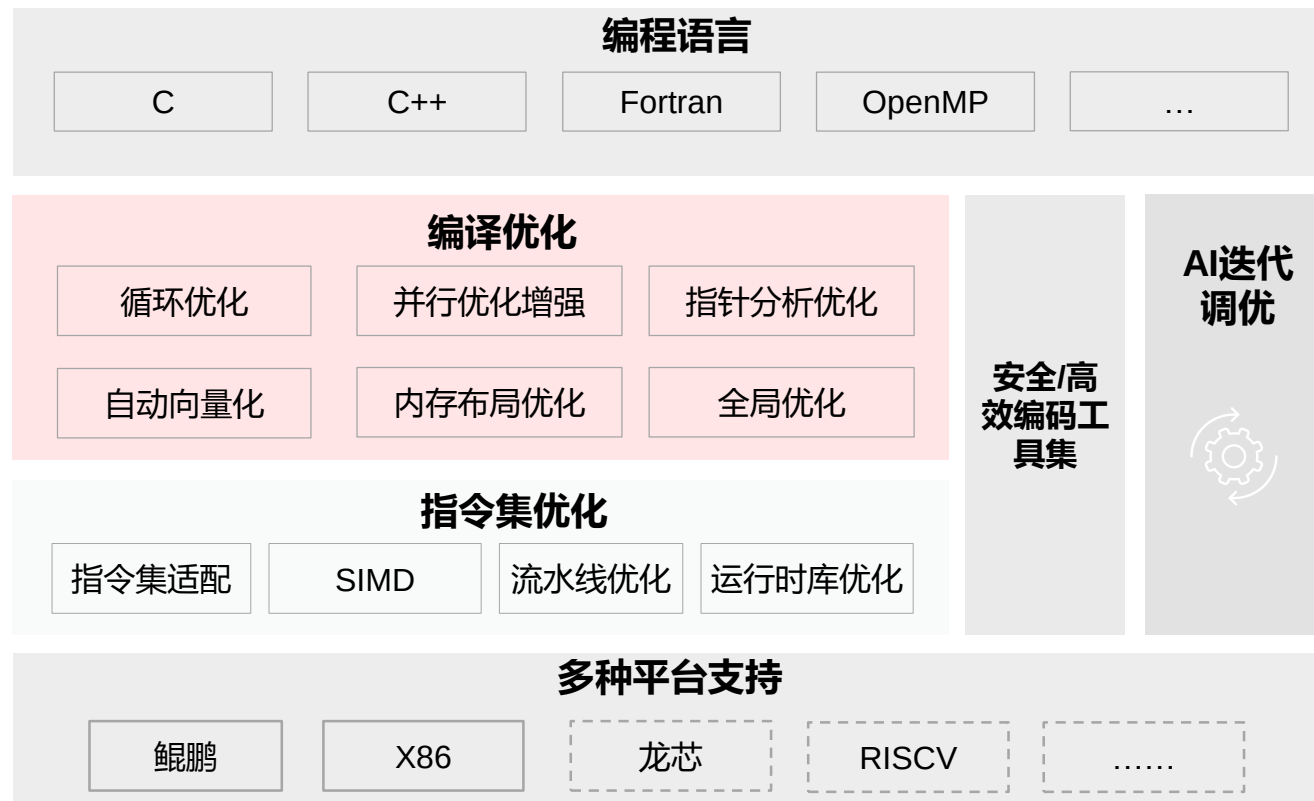


□ 基于LLVM扩展

- 支持C/C++/Fortran编程语言
- 增强中端编译优化技术
- 支持鲲鹏920、X86-64等硬件

□ 打造高性能、高可信、易扩展的编译工具链

- 性能/代码体积优化选项
- 多样化浮点精度选项/模式调优
- 静态检查工具，重构工具
- 支持AI迭代调优，自动优化编译配置
- 针对通用处理器架构的各类高性能编译优化技术





主要内容

- 1 编程语言及设计
- 2 编译器及形式
- 3 编译器的阶段
- 4 **编译技术的应用与挑战**



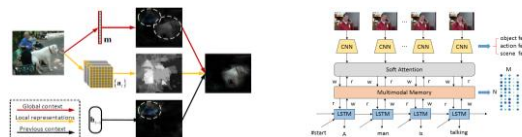
人工智能应用及深度学习框架

无人驾驶系统的软件栈

应用层



模型层



框架层



高层
中间表示

NNVM
Relay



编译

算子
实现层



优化

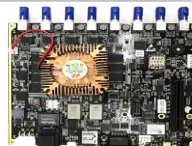
硬件层



Nvidia Drive
PX2



Nvidia Jetson
Nano



地平线“征途2.0”

国产系统软件与硬件



CANN

Compute
Architecture
for Neural
Networks

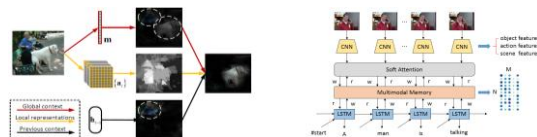


无人驾驶系统的软件栈

应用层



模型层



框架层



高层
中间表示

NNVM
Relay



编译

算子
实现层



优化

硬件层



Nvidia Drive PX2



Nvidia Jetson Nano



地平线 "征程2.0"

国产系统软件与硬件

科学计算应用/人工智能应用

编程框架/库/中间件

SWTVM SWTensorFlow SWPyTorch SWMind 应用平台基础框架

人工智能算子库 基础数学库 SWBlas SWSparse Shentu

编程语言

C C++ Fortran Python MPI OpenACC 并行C

基础组件

众核基础编译器 Athread运行时 动态运行支撑

支持SACA的神威平台

"神威"系列超级计算机

"神威"众核服务器



大模型时代的编译力

□ 编译力

运用编译原理的**抽象、分析、变换与生成方法**，
面向**程序、数据、模型**及**自然语言**等**广义计算对象**，
实现从**计算意图**到**可靠、高效**计算系统的**理解、构造、优化和验证**。

□ 编译内涵的拓展

- 传统编译：程序 → 程序
- 智能时代：多模态意图/知识/模型 → 可执行系统

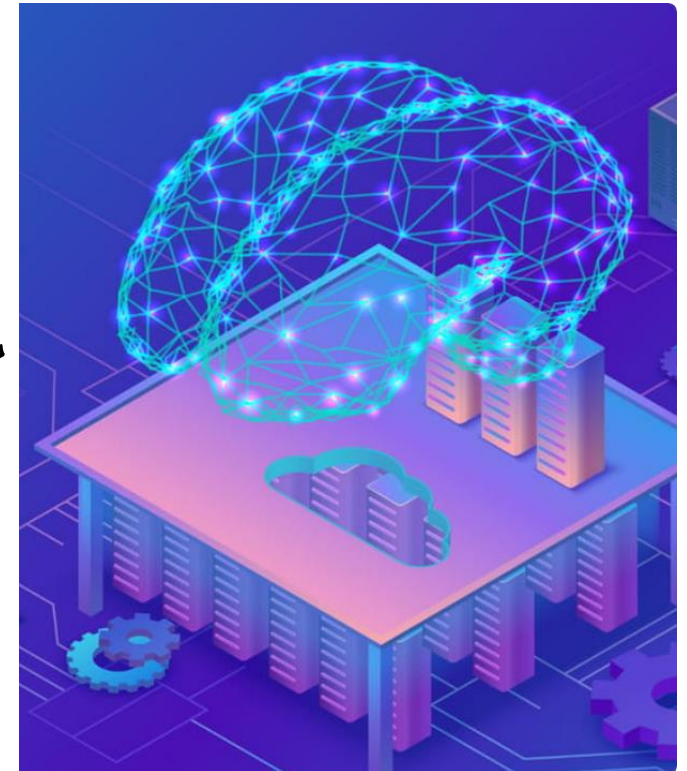
□ 编译力培养目标

编译器实现能力 → 智能系统构造能力

编译力 ≠ AI编程能力

编译力决定能否理解、约束、验证和优化AI生成的程序与系统

张昱：大模型时代的编译力培养





我听到的会忘掉，
我看到的能记住，
我做过的才真正明白。